Implementation and Performance Analysis of Parallel LU Factorization using OpenMP

Karthik Puranam (AE22B006) Abhinav Singh (AE22B005)

May 5, 2026

Abstract

This report presents the design, implementation, and performance evaluation of a parallel LU factorization solver using OpenMP on shared-memory multi-core CPUs. Three implementations are developed and benchmarked: a sequential baseline, a naive parallel version using loop-level `#pragma omp parallel for`, and a blocked (tiled) parallel version that exploits both cache locality and coarse-grained task parallelism.

1 Introduction

Solving large systems of linear equations $Ax = b$ is central to many scientific and engineering applications, including the numerical simulation of heat transfer, structural mechanics, and fluid dynamics. Direct methods such as LU decomposition are widely used due to their numerical stability and predictable cost.

However, standard LU decomposition has $O(N^3)$ computational complexity, which becomes prohibitively expensive for large matrices. Modern multi-core processors offer significant parallel compute capacity, yet naive parallelization often fails to extract meaningful speedup due to memory bandwidth limitations and poor cache utilization.

In our project, we look at one such application—the numerical solution to the 2D heat equation. We generate the matrix needed to solve an Euler implicit scheme and perform our blocked LU implementation to observe the speedups we can achieve.

We analyzed the performance based on factors like matrix size ($N \in \{484, 1024, 2025, 4096\}$), block size ($b \in \{64, 128, 256\}$), and Number of threads ($p \in \{1, 2, 4, 8\}$).

2 Mathematical Formulation

2.1 The 2D Heat Equation

The two-dimensional, time-dependent heat equation is:

$$\frac{\partial u}{\partial t} = \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) \tag{1}$$

where $u(x, y, t)$ is the temperature field and α is the thermal diffusivity.

2.2 Implicit Finite Difference Discretization

Using a fully implicit backward-Euler scheme on a uniform $n \times n$ grid with spacing h and time step Δt, and defining $\lambda = \alpha \Delta t / h^2$:

$$(I - \lambda A_h)\, u^{t+1} = u^t \tag{2}$$

where A_h is the 5-point Laplacian stencil matrix. This yields a linear system

$$A\, x = b, \qquad A = I - \lambda A_h \tag{3}$$

of size $N = n^2$. The matrix A is sparse, symmetric positive definite, and has a block tridiagonal structure. Fill-in during factorization makes the resulting L and U matrices dense.

3 Algorithm Design

3.1 Blocked (Tiled) LU Decomposition

The matrix is partitioned into $b \times b$ tiles. For each diagonal tile at block index kk, the algorithm proceeds in four steps:

Algorithm 1 Blocked LU Decomposition

1: **for** $kk = 0$ to $N_b - 1$ **do** ▷ $N_b = \lceil n/b \rceil$ block columns
2: **Step 1:** Factor diagonal tile $A[kk, kk]$ sequentially
3: **Step 2: (parallel)** Update column panel: tiles $A[ii, kk]$ for $ii > kk$
4: **Step 3: (parallel)** Update row panel: tiles $A[kk, jj]$ for $jj > kk$
5: **Step 4: (parallel, collapse 2)** Update trailing submatrix: tiles $A[ii, jj]$ for $ii, jj > kk$

$$A[ii, jj] \mathrel{-}= A[ii, kk] \cdot A[kk, jj]$$

6: **end for**

Step 4 is the dominant computation and exposes $O(N_b^2)$ independent tile updates, enabling excellent parallelism.

3.2 Time and Space Complexity Analysis

Sequential & Naive LU: The standard factorization requires computing elements across nested loops. The total number of floating-point operations is $\approx \frac{2}{3}N^3$, resulting in a time complexity of $O(N^3)$. Space complexity is $O(N^2)$ to store the dense matrix fill-in. Naive parallelization theoretically reduces time to $O(N^3/p)$ for p threads, but introduces $O(N \cdot p \log p)$ synchronization overhead due to barriers at every scalar step k.

Blocked LU: The blocked algorithm maintains the $O(N^3)$ computational time complexity. However, its **I/O (memory transfer) complexity** is drastically improved. Standard LU transfers $O(N^3)$ data between main memory and cache. Blocked LU loads a $b \times b$ block into the cache and performs $O(b^3)$ operations on it, reducing the total memory transfers to $O(N^3/b)$. Furthermore, barrier synchronization overhead is reduced by a factor of b, from N barriers down to $N_b = \lceil N/b \rceil$ barriers.

4 OpenMP Parallelization Strategy

4.1 Directives Used

Listing 1: Key OpenMP directives in blocked LU

```
// Step 2: column panel      independent per row-block
#pragma omp parallel for schedule(dynamic, 1)
for (int ii = kk+1; ii < nb; ++ii) { /* ... */ }

// Step 3: row panel      independent per column-block
#pragma omp parallel for schedule(dynamic, 1)
for (int jj = kk+1; jj < nb; ++jj) { /* ... */ }

// Step 4: trailing submatrix      all (ii, jj) pairs are independent
#pragma omp parallel for schedule(dynamic, 1) collapse(2)
for (int ii = kk+1; ii < nb; ++ii)
  for (int jj = kk+1; jj < nb; ++jj)
```

4.2 Amdahl's Law Analysis (For N=4096, b=128)

The sequential fraction f is dominated by Step 1 (diagonal tile factorization) at each of N_b iterations. For $N = 4096$, $b = 128$, $N_b = 32$:

$$f \approx \frac{N_b \cdot (b^3/3)}{2N^3/3} = \frac{N_b}{2(N/b)^3} = \frac{32}{2 \cdot 32^3} = \frac{1}{2048} \approx 0.00048 \tag{4}$$

Theoretical maximum speedup: $S_{\max} = 1/f \approx 2048\times$. In practice, hardware constraints like memory bandwidth heavily restrict this.

5 Results and Discussion

5.1 Execution Time vs Matrix Size (Fixed Block Size = 128)

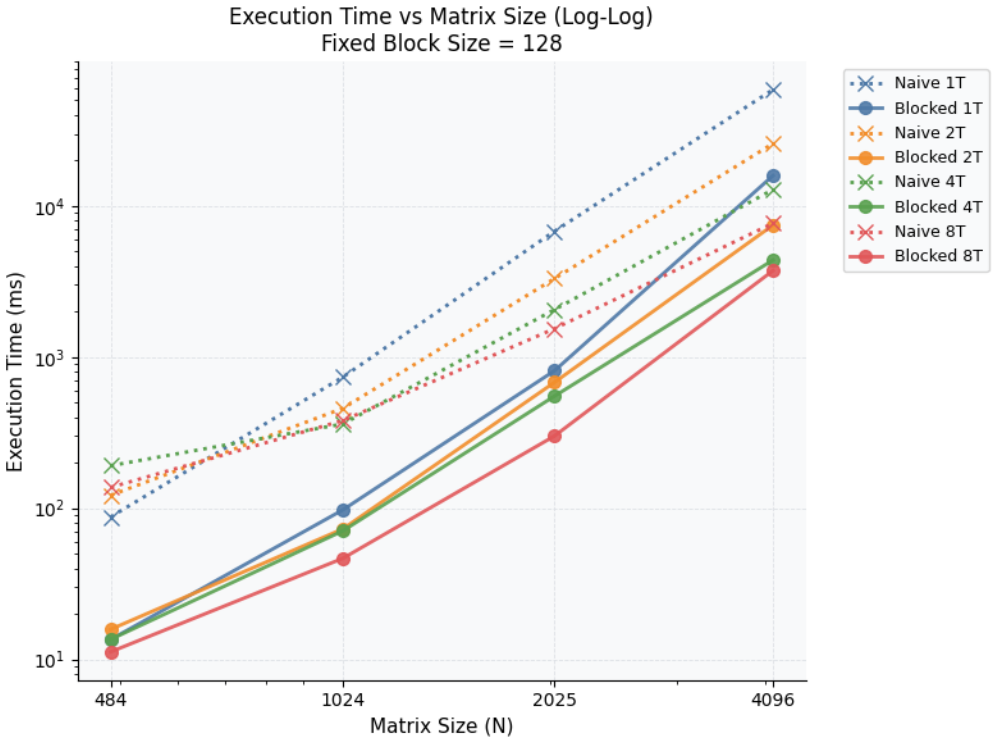

Figure 1: Execution time (log-log scale) vs matrix size for a fixed block size of 128. All implementations exhibit $O(N^3)$ growth (slope of ≈ 3).

The log-log plot demonstrates the empirical $O(N^3)$ time complexity for both Naive and Blocked implementations. The Blocked solver is consistently faster than the Naive solver across all matrix sizes. For $N = 4096$ using 8 threads, the Naive time is ≈ 7680 ms, while the Blocked time is ≈ 3733 ms. Furthermore, at small matrix sizes ($N = 484$), the 8-thread Naive approach is actually slower than the 1-thread Naive approach due to thread management and barrier overheads dominating the sparse computational work.

5.2 Speedup vs Matrix Size (Fixed Block Size = 128)

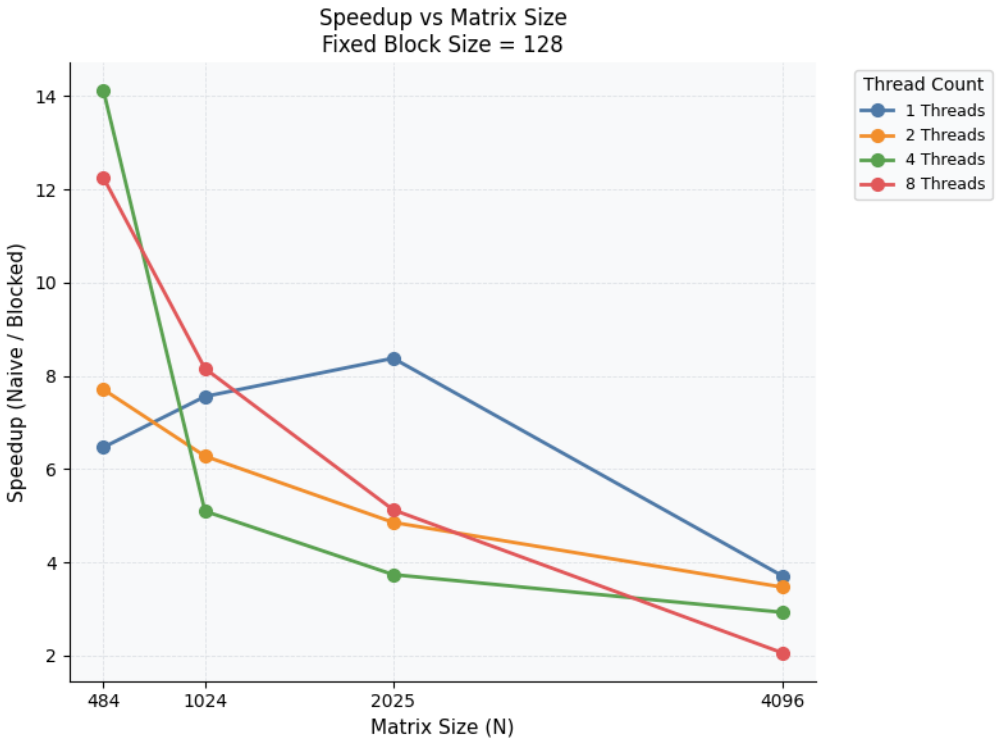

Figure 2: Execution time (log-log scale) vs matrix size for a fixed block size of 128. All implementations exhibit $O(N^3)$ growth (slope of ≈ 3).

This plot reveals an interesting dynamic regarding the *relative* speedup (Naive Time / Blocked Time). At smaller matrices ($N = 484$), the relative speedup for 8 threads is massive ($\approx 12.2\times$). This is not just because Blocked is fast, but because Naive is highly inefficient at small sizes due to synchronization overhead. As N approaches 4096, the relative speedup settles to $\approx 2.06\times$ (for 8 threads). This occurs because the Naive implementation begins to saturate the cores efficiently with enough work to amortize the barrier costs, while the Blocked version eventually encounters the memory bandwidth ceiling of the shared-memory architecture.

5.3 Execution Time vs Block Size ($N = 4096$)

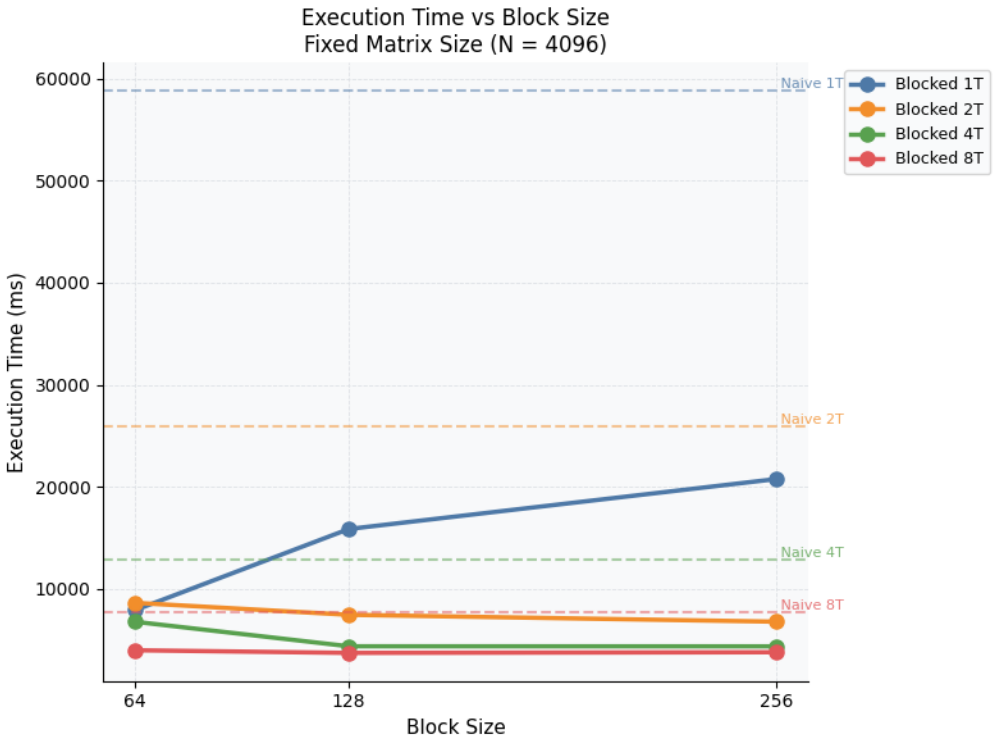

Figure 3: Execution time (log-log scale) vs matrix size for a fixed block size of 128. All implementations exhibit $O(N^3)$ growth (slope of ≈ 3).

Evaluating at the largest matrix size ($N = 4096$), we observe the direct impact of the block parameter (b). The execution times for the Blocked implementations are relatively stable across $b \in \{64, 128, 256\}$. However, $b = 128$ and $b = 256$ slightly outperform $b = 64$ across all thread counts. This confirms that a block size of 128 (occupying roughly 128 KB, fitting comfortably in L2 cache) strikes an optimal balance between minimizing memory transfers and providing enough coarse-grained tiles for load balancing.

5.4 Speedup vs Block Size ($N = 4096$)

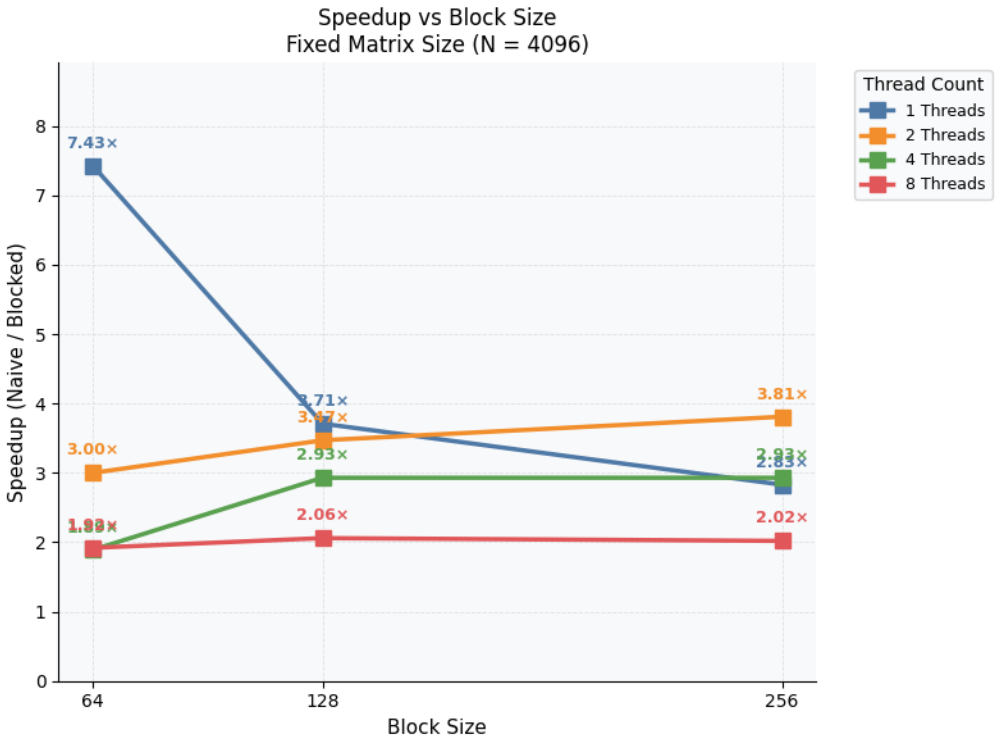

Figure 4: Execution time (log-log scale) vs matrix size for a fixed block size of 128. All implementations exhibit $O(N^3)$ growth (slope of ≈ 3).

The speedup chart at $N = 4096$ explicitly quantifies the performance multiplier. For a single thread ($1T$), using a block size of 128 yields a 3.71× speedup over the 1T Naive implementation. This isolated metric proves that memory locality optimization (blocking) provides massive performance gains even without parallelization.When scaled to 8 threads, the Blocked version (bs=128) achieves a 2.06× speedup over the 8T Naive version.

6 Conclusion

- **Blocking Without Threads:** At $N = 4096$, a single-threaded Blocked LU (bs=128) completes in $\approx$ 15.8s compared to $\approx$ 58.8s for the single-threaded Naive LU. This 3.71× speedup is achieved purely through cache reuse, proving that memory optimization is a fundamental prerequisite to parallel scaling.

- **Synchronization Overhead:** Naive parallelization (loop-level `parallel for`) is highly detrimental for smaller matrices ($N \leq 484$). The barrier cost per k-step exceeds the parallel benefit, making multi-threaded Naive LU slower than its single-threaded counterpart.

- **Absolute Performance:** The optimal configuration (Blocked, bs=128, 8 threads) solved the 4096×4096 system in just 3.73 seconds, representing a 15.75× absolute speedup over the sequential (1T Naive) baseline.

- **Comparison to Industry Standards:**
  Ultimately, our best configuration achieved a 15.75× speedup (dropping the execution time from 58.8 seconds to 3.73 seconds for $N = 4096$) over the basic sequential version. While

this highlights the massive benefit of combining cache blocking with OpenMP, industry-standard libraries like Intel MKL or OpenBLAS are still significantly faster. They achieve this by using *hierarchical* blocking to optimize for all cache levels (registers, L1, L2, and L3), rather than just the L2 cache like we did. They also rely on highly tuned SIMD vector instructions (like AVX2) and use partial pivoting to guarantee stability. Even so, our implementation successfully demonstrates the main takeaway: in high-performance computing, managing memory efficiently is just as important as simply adding more threads.